ПРОГРАММА НА AVR ASSEMBLER

Два расписания на одном счётчике

Задача программы - периодически передавать две строки через последовательный порт и принимать команды для изменения их текста и частоты. Начальные сообщения: **ping** через каждые 2 мс и **pong** через каждые 3 мс.

Принцип работы

Timer1 непрерывно считает такты процессора. В двух регистрах сравнения, OCR1A и OCR1B, хранятся ближайшие сроки отправки. Когда счётчик достигает одного из сроков, возникает прерывание соответствующего канала.

Обработчик не ждёт передачи строки. Он назначает следующий срок и ставит номер сообщения в очередь. Основной цикл извлекает номера по порядку и отправляет нужные тексты. Так события таймера отделены от работы единственного передатчика USART.

Пример расписания

При старте от общего момента первый канал создаёт события на 2, 4 и 6 мс, второй - на 3 и 6 мс. На 6 мс возникают два события. Они поступают в очередь и обслуживаются последовательно; строки не смешиваются побайтно.

Параметры реализации

Микроконтроллер ATmega8 работает на реальной частоте 8 МГц. Timer1 используется без предделителя. USART настроен на 500000 бод, 8N1; RX - PD0, TX - PD1. Очередь вмещает 63 ожидающих события.

Основной файл: **serial_scheduler/serial_scheduler.asm**. Собранный образ находится рядом, в serial_scheduler.hex. Названия on_channel_one, queue_event и dispatch_loop обозначают обработчик первого канала, постановку события и основной цикл.

Постановка задачи: Achievement #1, слайд 5 первой лекции. Исходный код и протокол команд разработаны для работы; сведения о регистрах взяты из документации ATmega8.

АЛГОРИТМ И КОМАНДЫ

Как устроена программа

Назначение следующего события

Срок вычисляется от предыдущего срока, а не от момента входа в обработчик. Поэтому задержка выполнения обработчика не прибавляется к каждому следующему периоду. Ниже приведён фрагмент исходника.

```
on_channel_one:
 SAVE
 in r18,OCR1AL
 in r19,OCR1AH
 lds r16,channel_one_period
 lds r17,channel_one_period+1
 add r18,r16
 adc r19,r17
 out OCR1AH,r19
 out OCR1AL,r18
 ldi r16,1
 rcall queue_event
 RESTORE
```

Период занимает два байта. Команды add и adc складывают младшую и старшую части с учётом переноса. После записи нового срока в OCR1A номер канала 1 передаётся функции queue_event.

Команды управления

T1 4000 и **T2 6000** задают периоды каналов. Четыре HEX-цифры означают такты процессора: 0x4000 = 16384 такта = 2,048 мс. **S1 hello** и **S2 world** заменяют тексты. **R** начинает два расписания от общего текущего момента, **?** возвращает настройки.

Каждая команда заканчивается LF. Следующую отправляют после ответа !OK или !ERR. Текст содержит до 16 печатных ASCII-символов и не начинается с !. Завершение CRLF добавляется автоматически. Неудачная команда оставляет прежние настройки.

Период меняется при назначении следующего события; ближайший уже назначенный срок сохраняется. Команда R очищает ожидающие события, но не останавливает и не обнуляет аппаратный счётчик.

РЕЗУЛЬТАТ И ВОСПРОИЗВЕДЕНИЕ

Проверка и границы решения

Что проверено

После переименования исходник заново собран AVRA и выполнен в simavr. Все **31 сценарий** прошли: изменение периодов и строк, перезапуск, чтение настроек, ошибочные команды и недопустимые значения. Получено **957 полных строк** без перемешивания. Остановка счётчика после запуска не наблюдалась.

Размер программы - 1368 байт кода и 280 байт объявленных данных; стек учитывается отдельно. Полный результат проверки: serial_scheduler/checks/result.json.

Допустимая нагрузка

Для периода T и длины текста L применяется ограничение **max(4096, (L + 2) × 1024) ≤ T ≤ 65535**. Для ping/pong нижняя граница равна 6144 тактам, сумма двух периодов - 12288 тактам. Это выбранный диапазон с запасом, а не точный поиск всех рабочих настроек или глобального минимума.

Невыполненное требование

Начало передачи с точностью четыре такта не обеспечено. Совпавшие события попадают в очередь, но один USART не может одновременно начать две строки. Передача ping с CRLF занимает 960 тактов при выбранной скорости. Событие сравнения и физическое начало передачи - разные моменты.

Проверка выполнена в симуляторе. Его UART-callback показывает запись регистра UDR, а не фронт на TX. На плате программа не испытывалась. Высокий уровень не подтверждён: требуется согласовать смысл точности. Расчёт конфликта: checks/timing_audit.py.

Как повторить проверку

```
cd serial_scheduler
python3 build_and_check.py
```

Скрипт собирает фиксированные версии AVRA и simavr в build/. Нужны Python 3, git, make, C-компилятор и доступ к GitHub. Для общения с платой используется send_command.py и pyserial 3.5; подсказка: python3 send_command.py --help.

Технический источник: ATmega8(L) Complete Datasheet, разделы Timer/Counter1 и USART.